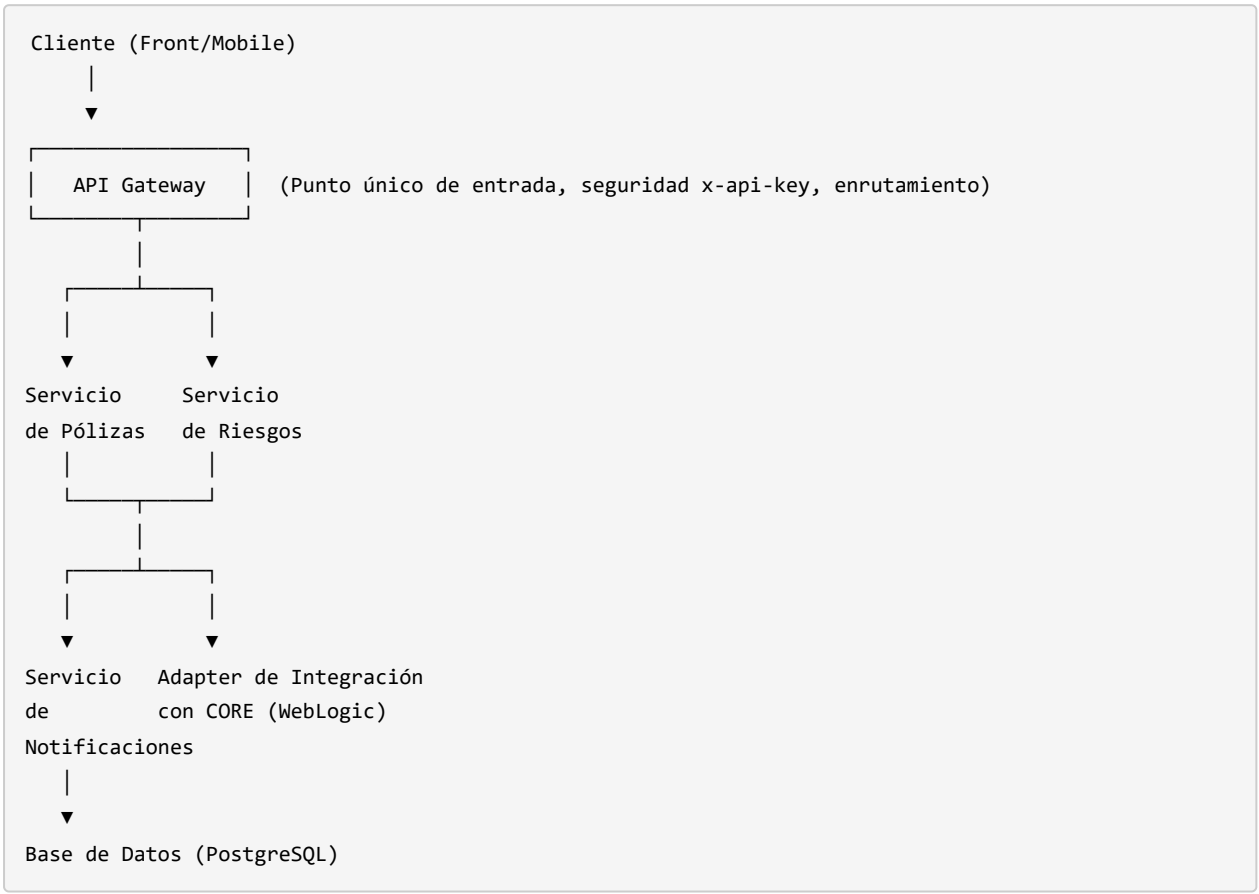
Módulo 1 – Diseño de Sistema

1. Arquitectura

La plataforma sigue una arquitectura en capas con separación de responsabilidades:



Componentes principales

- **API Gateway:** Entrada unificada, autenticación básica (header x-api-key), enrutamiento a servicios backend.
- **Servicio de Pólizas:** Gestión del ciclo de vida de pólizas (crear, consultar, modificar, renovar, cancelar).
- **Servicio de Riesgos:** Gestión de riesgos asociados a pólizas colectivas (agregar, cancelar, consultar).
- **Servicio de Notificaciones:** Envío de eventos de correo y SMS (creación y renovación de pólizas).
- **Adapter de Integración con CORE:** Capa en WebLogic que expone un servicio agnóstico de edición. Toda modificación de estado de póliza o riesgo debe pasar por este adapter para mantener sincronizado el CORE legado.

- **Base de Datos:** Almacenamiento persistente de pólizas, riesgos, clientes y eventos.

2. Patrones de Arquitectura Seleccionados

a) API Gateway

Justificación: Centraliza el punto de entrada de la plataforma, permite aplicar seguridad de forma uniforme (validación del header x-api-key: 123456), enrutar peticiones a los servicios correspondientes (Pólizas, Riesgos, Notificaciones) y desacoplar al cliente de la arquitectura interna de microservicios. Facilita además el versionamiento de APIs y el control de tráfico.

b) Event-Driven (Arquitectura Orientada a Eventos)

Justificación: Desacopla la creación y renovación de pólizas del envío de notificaciones (correo/SMS) y de la sincronización con el CORE. Cuando se crea o renueva una póliza, el servicio publica un evento; el Servicio de Notificaciones y el Adapter de CORE lo consumen de forma asíncrona. Esto mejora la resiliencia y la disponibilidad 24/7, ya que una caída temporal del servicio de correo o del CORE no bloquea la operación principal.

c) Hexagonal (Ports & Adapters)

Justificación: Aísla el núcleo de negocio (reglas de pólizas, renovación con IPC, cancelación) de las tecnologías externas (WebLogic/CORE, base de datos, API REST). El servicio agnóstico de edición de WebLogic se implementa como un *adapter* externo que se conecta a través de un *puerto* definido por el negocio. Esto permite cambiar el CORE legado o la capa media sin modificar el core de la aplicación.

3. Modelo de Datos Principal

Entidades clave

- **Póliza:** identificador, tipo (Individual/Colectiva), estado (Activa/Cancelada/Renovada), fecha de inicio, fecha de fin, canon mensual, valor de prima, IPC aplicado, identificador del tomador/arrendatario, identificador del beneficiario/arrendador.
- **Riesgo:** identificador, identificador de póliza asociada, descripción del riesgo, estado, monto asegurado, fecha de alta, fecha de cancelación (si aplica).
- **Persona/Cliente:** identificador, tipo (arrendatario/arrendador), nombre, datos de contacto (correo, teléfono), rol en la póliza.
- **Vigencia / Renovación:** identificador de póliza, periodo de vigencia, número de renovación, fecha de renovación, valor de canon antes y después del ajuste

IPC.

- **Notificación:** identificador, tipo de evento (creación/renovación), canal (correo/SMS), destinatario, fecha de envío, estado.
- **Evento CORE:** identificador, tipo de operación (ACTUALIZACIÓN/CANCELACIÓN), identificador de póliza, fecha de intento de envío, estado del envío al CORE.

Reglas del modelo

- Una póliza **Individual** tiene exactamente 1 riesgo.
- Una póliza **Colectiva** tiene 1 o muchos riesgos.
- La prima se calcula como: canon mensual × número de meses de vigencia.
- Al renovar, se ajusta el canon según el incremento del IPC y se conserva el mismo periodo de vigencia inicial.

4. Estrategias Operativas

Escalabilidad

- Despliegue de los servicios en contenedores (Docker) con orquestador (Kubernetes) para escalar horizontalmente según carga.
- API Gateway con balanceo de carga.
- Base de datos PostgreSQL con réplicas de lectura para consultas (GET /polizas) y escritura primaria para transacciones.
- Índices adecuados en columnas de filtrado frecuente (tipo, estado, fecha de vigencia).

Logs y Observabilidad

- Logs estructurados (JSON) en todos los servicios con identificador de correlación (correlation ID) para rastrear peticiones end-to-end.
- Métricas clave: latencia de endpoints, tasa de error, volumen de renovaciones y cancelaciones.
- Trazabilidad de eventos enviados al CORE y notificaciones despachadas.
- Health checks en /actuator/health para monitoreo de disponibilidad 24/7.

Tolerancia a Fallos

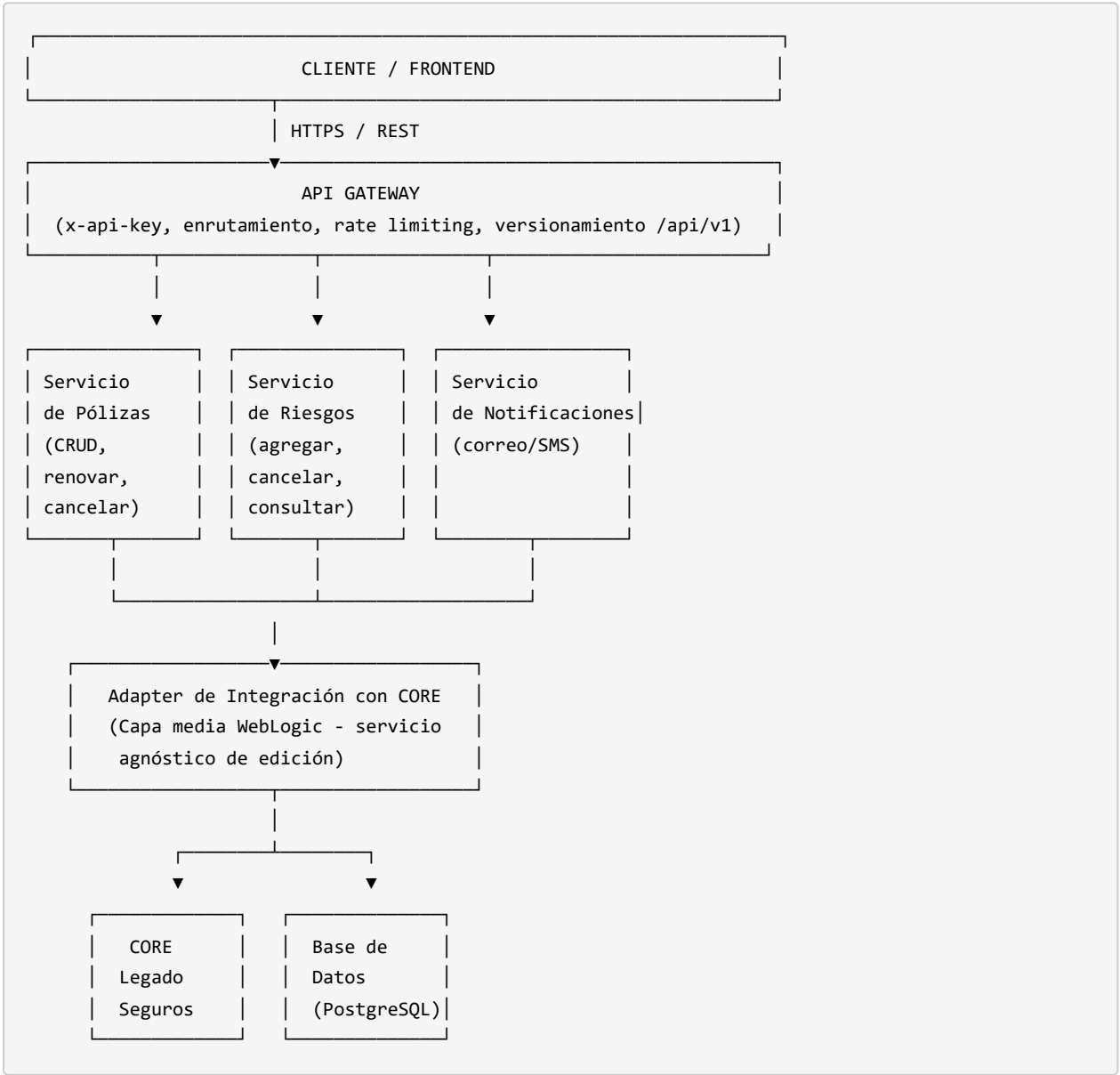
- Reintentos con backoff exponencial para llamadas al Adapter de CORE y servicio de notificaciones.
- Circuit breaker para evitar cascada de fallos si el CORE legado está caído.

- Cola de mensajes (o registro persistente de eventos) para no perder operaciones de notificación ni sincronización con CORE ante caídas temporales.
- Transacciones de base de datos para garantizar consistencia al cancelar una póliza y sus riesgos asociados.

Versionamiento de APIs

- Versionado en la URL: /api/v1/polizas, /api/v1/riesgos.
- Política de retrocompatibilidad: mantener al menos una versión anterior activa durante un periodo de transición.
- Documentación de cambios (changelog) entre versiones.
- El header x-api-key se valida independientemente de la versión.

5. Diagrama de Componentes



Supuestos de Diseño

- La seguridad mínima se implementa mediante header x-api-key: 123456 validado en el API Gateway o en un filtro de Spring Boot.
- El CORE legado se considera un sistema externo no disponible para pruebas directas; se simula con el endpoint /core-mock/evento.
- La base de datos relacional (PostgreSQL) es suficiente para el alcance de la prueba; no se requiere NoSQL.